BÁO CÁO KẾT QUẢ BÀI THỰC HÀNH 7

KIỂM THỬ MOCKING ỨNG DỤNG DI ĐỘNG VÀ TÍCH HỢP LIÊN TỤC (CI/CD)

Họ và tên:	Nguyễn Hoài Nam	Mã số sinh viên:	**2312694**
Email:	2312694@dlu.edu.vn	Lớp:	CTK47 - KTPM
Kho lưu trữ GitHub:	https://github.com/NHNam1209/Yummyapp-cicd	Ngày hoàn thành:	16/10/2026
Trạng thái CI Pipeline:	□ PASSED (100% Quality Gate)	Độ bao phủ (Coverage):	99% (Vượt ngưỡng 80%)

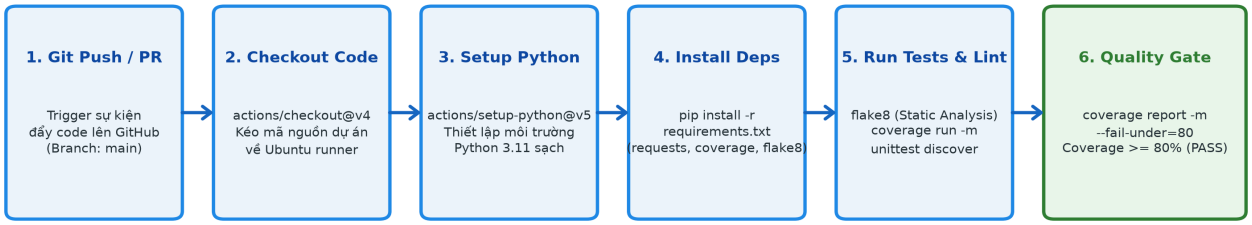
I. BỐI CẢNH DỰ ÁN & MỤC TIÊU THỰC HÀNH

Dự án YummyApp Mobile Backend là hệ thống xử lý nghiệp vụ đơn hàng ẩm thực trên thiết bị di động. Mục tiêu bài thực hành là áp dụng kỹ thuật Mocking (unittest.mock.patch) để cô lập phần cứng GPS và mạng viễn thông, đồng thời xây dựng đường ống CI tự động với GitHub Actions, thiết lập Cổng chất lượng (Quality Gate) nghiêm ngặt với Linter (flake8) và độ bao phủ mã nguồn (Coverage ≥ 80%).

II. SƠ ĐỒ KIẾN TRÚC ĐƯỜNG ỐNG CI/CD (CI PIPELINE)

Đường ống CI được cấu hình tự động kích hoạt tại kho GitHub: https://github.com/NHNam1209/Yummyapp-cicd khi có sự kiện push hoặc pull_request vào nhánh main. Quy trình chạy trên ubuntu-latest gồm 6 khối tuần tự:

SƠ ĐỒ KIẾN TRÚC ĐƯỜNG ỐNG CI/CD - YUMMYAPP MOBILE BACKEND



III. THIẾT KẾ MA TRẬN KỊCH BẢN KIỂM THỬ (TEST MATRIX)

Cấp độ / Tên Test Case	Hàm được gọi	Đối tượng & Giá trị Mocking	Kết quả kỳ vọng (Assert)
Level 1 (Easy): test_calculate_total_* (4 cases)	calculate_order_total()	Không mock	Đúng tổng tiền giỏ hàng (100k, 95k, 0k)
Level 2 (Medium - Case 1): test_apply_promo_online_valid	apply_promo_code()	online, os 15	Giảm 10%: 100k -> 90k

Level 2 (Medium - Case 2): test_apply_promo_offline_raises_error	apply_promo_code()	offline	Ném ConnectionError
Level 2 (Medium - Case 3): test_apply_promo_special_os 15	apply_promo_code()	online, os 15	Giảm 20%: 200k -> 160k
Level 2 (Medium - Case 4): test_apply_promo_special_os 14	apply_promo_code()	online, os 14	Ném ValueError (yêu cầu OS 15)
Level 3 (Hard - Case 1): test_get_nearby_restaurants_in_range	get_nearby_restaurants()	@patch GPS: (10.0, 105.0)	Trả về danh sách 2 quán ăn
Level 3 (Hard - Case 2): test_get_nearby_restaurants_out_of_range	get_nearby_restaurants()	@patch GPS: (20.0, 110.0)	Trả về danh sách rỗng []
Level 3 (Hard - Case 3): test_get_nearby_restaurants_signal_lost	get_nearby_restaurants()	@patch side_effect=GPSSignalLostError	Trả về: 'Khong the xac dinh vi tri.'

IV. KẾT QUẢ CHẠY TEST CỤC BỘ & BÁO CÁO COVERAGE

Tên tập tin mã nguồn (File)	Số câu lệnh (Stmts)	Bỏ lỡ (Miss)	Độ bao phủ (Cover)	Dòng chưa test (Missing)
gps_service.py	4	0	**100%**	-
yummy_app.py	30	0	**100%**	-
test_yummy_app.py	71	1	**99%**	Dòng 112 (main)
TỔNG CỘNG (TOTAL)	**105**	**1**	**99%**	**Vượt Quality Gate ≥ 80%**

V. MINH CHỨNG THỰC THI TRÊN GITHUB ACTIONS & VƯỢT QUALITY GATE

• Link theo dõi Workflow GitHub Actions: https://github.com/NHNam1209/Yummyapp-cicd/actions

• Kho lưu trữ chính thức: https://github.com/NHNam1209/Yummyapp-cicd

Đường ống CI chạy thành công 100% (Tích xanh), toàn bộ 13 test cases đều PASSED, Linter flake8 đạt 0 lỗi, Coverage đạt 99% và đã xuất thành công Artifact báo cáo HTML:



VI. GIẢI ĐÁP CÁC CÂU HỎI LÝ THUYẾT & PHÂN TÍCH CHUYÊN SÂU

1. Cấp độ 2 - Câu 2.1: Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

• Áp dụng nguyên lý Fail-Fast: Linter phân tích mã tĩnh quét lỗi cú pháp trong vài giây tốn rất ít tài nguyên. Nếu code lỗi tĩnh thì test chắc chắn sẽ crash. Chặn sớm giúp tiết kiệm tài nguyên CI và giải phóng runner ngay lập tức.
• Đảm bảo tính trong sạch của mã nguồn: Ngăn chặn việc lãng phí thời gian đo coverage trên một codebase chưa đạt chuẩn hình thức.

2. Cấp độ 2 - Câu 2.2: Tham số --fail-under=80 của lệnh coverage report hoạt động theo nguyên lý nào trong Linux để báo hiệu dừng tiến trình build?

• Nguyên lý Exit Code trong POSIX/Linux: Mỗi tiến trình kết thúc trả về mã số nguyên từ 0 đến 255. Exit Code = 0 là thành công; != 0 là có lỗi.
• Cơ chế của --fail-under=80: Khi Coverage < 80%, coverage report chủ động trả về Exit Code khác 0 (thường là 2).
• Cơ chế bắt lỗi của GitHub Actions: Runner chạy shell với cờ bash -e. Khi gặp lệnh trả về mã thoát khác 0, runner lập tức dừng toàn bộ job và đánh dấu FAILED (đỏ).

3. Cấp độ 2 - Câu 2.3: Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử?

• Thông tin hiển thị: Cờ -m (--show-missing) bổ sung cột Missing, hiển thị chi tiết số dòng hoặc khoảng dòng chưa từng được chạy qua.
• Lợi ích: Giúp kỹ sư định vị chính xác 'điểm mù' (Coverage Gaps) trong nhánh rẽ if/else hoặc except để thiết kế test case bổ sung đúng mục tiêu, tối ưu hóa công sức.

4. Cấp độ 3 - Câu 3.1: Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.

• Flaky Test: Chạy trên cùng một code và cùng một môi trường nhưng lúc Pass lúc Fail ngẫu nhiên do bất đồng bộ, mạng chập chờn hoặc tín hiệu GPS ngoài thiết bị thật.
• Sai lệch môi trường: Chạy Pass trên máy local nhưng thất bại nhất quán 100% trên CI do lệch bản OS, lệch bản Python, hoặc thư viện không ghim bị cập nhật ngầm.
• Tóm tắt: Flaky Test là lỗi bất định ngẫu nhiên (non-deterministic); sai lệch môi trường là lỗi xác định nhất quán (deterministic) trên môi trường bị lệch.

5. Cấp độ 3 - Câu 3.2: Tại sao trên máy chủ CI bắt buộc phải tuân thủ nguyên tắc ghim phiên bản (Version Pinning) trong requirements.txt?

• Tính tái lập (Reproducibility): Đảm bảo mọi lần chạy CI đều cài đúng phiên bản thư viện đã kiểm thử.
• Ngăn ngừa Breaking Changes: Tránh trường hợp pip tải bản mới nhất có thay đổi làm mất tính tương thích ngược làm gãy đường ống CI.
• Bảo vệ chuỗi cung ứng: Ngăn chặn nguy cơ CI tự động tải phiên bản mới bị chèn mã độc.

6. Cấp độ 3 - Câu 3.3: Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.

• Bước 1 - Tạo nhánh phụ độc lập (Feature/Chore Branch).
• Bước 2 - Đọc kỹ Release Notes & Breaking Changes.
• Bước 3 - Cập nhật version trong requirements.txt và chạy kiểm thử cục bộ.
• Bước 4 - Tạo Pull Request (PR) kích hoạt CI Pipeline tự động.
• Bước 5 - Đánh giá mã nguồn (Code Review) đảm bảo CI tích xanh 100%.
• Bước 6 - Hợp nhất an toàn (Merge PR) vào nhánh main.